TECHNICAL MANUAL

radcoolpv

Physics, Numerical Models, YAML Cases, and Validation

Project manual · August 30, 2026

Contents

Purpose and scientific status

radcoolpv calculates optical properties of periodic photonic structures for radiative cooling and couples them to a silicon PV thermal/electrical model. Live optics use only the Stanford S4 Python extension. S4 is imported lazily; reduced-spectrum and free-form readers can run without it.

S4 solves RCWA (rigorous coupled-wave analysis): the unit cell is laterally infinite and strictly periodic, layers are coherent, and the Fourier basis is truncated at s4_modes.

The package reports spectral reflectance R, transmittance T, total absorptance A, silicon-layer absorptance A_{Si}, and, under the conditions stated in Section 5.4, emittance. The thermal stage reports operating temperature, I–V characteristics, maximum-power-point output, efficiency, and a temperature reduction relative to a user-specified reference.

The library is not a universal PV module model. Its central assumptions are a periodic coherent optical unit cell, scalar material permittivities, a lumped steady-state thermal balance, and a single-diode silicon PV model. Validation status is evidence-labelled in Section 10; a passing software test is not treated as proof of physical validity.

Installation

```
./setup.sh
source ~/.venvs/radcoolpv-py/bin/activate
python -c "import radcoolpv; print(radcoolpv.__version__)"
```

setup.sh creates the environment outside the repository, at ~/.venvs/radcoolpv-py by default (override with VENV_DIR). A virtual environment placed inside an iCloud-synced folder such as ~/Documents or ~/Desktop is marked hidden by the operating system, and Python skips hidden .pth files, which silently disables the editable install. Inside the activated environment use python, not python3: a python3 alias in the shell profile shadows the environment even when it is active, and reports a missing dependency such as No module named 'yaml' instead of naming the real cause.

A stale radcoolpv on PATH—from an earlier pip install -e . into another interpreter—still resolves to the repository path recorded at that install, and reports No module named 'radcoolpv'. Check which -a radcoolpv.

Figure 1 lays out both supported entry points side by side: a local install and a zero-install Colab session.

The thermal/PV stage and spectrum readers use only the Python dependencies installed by setup.sh. Live optics additionally requires a separately compiled S4 Python module, pinned to a specific fork and revision:

```
brew install fftw suite-sparse openblas lapack boost
git clone https://github.com/phoebe-p/S4
cd S4
git checkout 9569f5e555b967a4324eb1ea593d0f9f40761a61
make -f Makefile.m1 S4_pyext
```

```
python -c "import S4; print(S4.__file__)"
```

The S4 import occurs only when a case with `geometry.source: s4` executes; other forks or revisions of S4 are not compatibility targets. Git is optional everywhere: without it the manifest records a null commit and the run proceeds.

Running the code: macOS and Colab

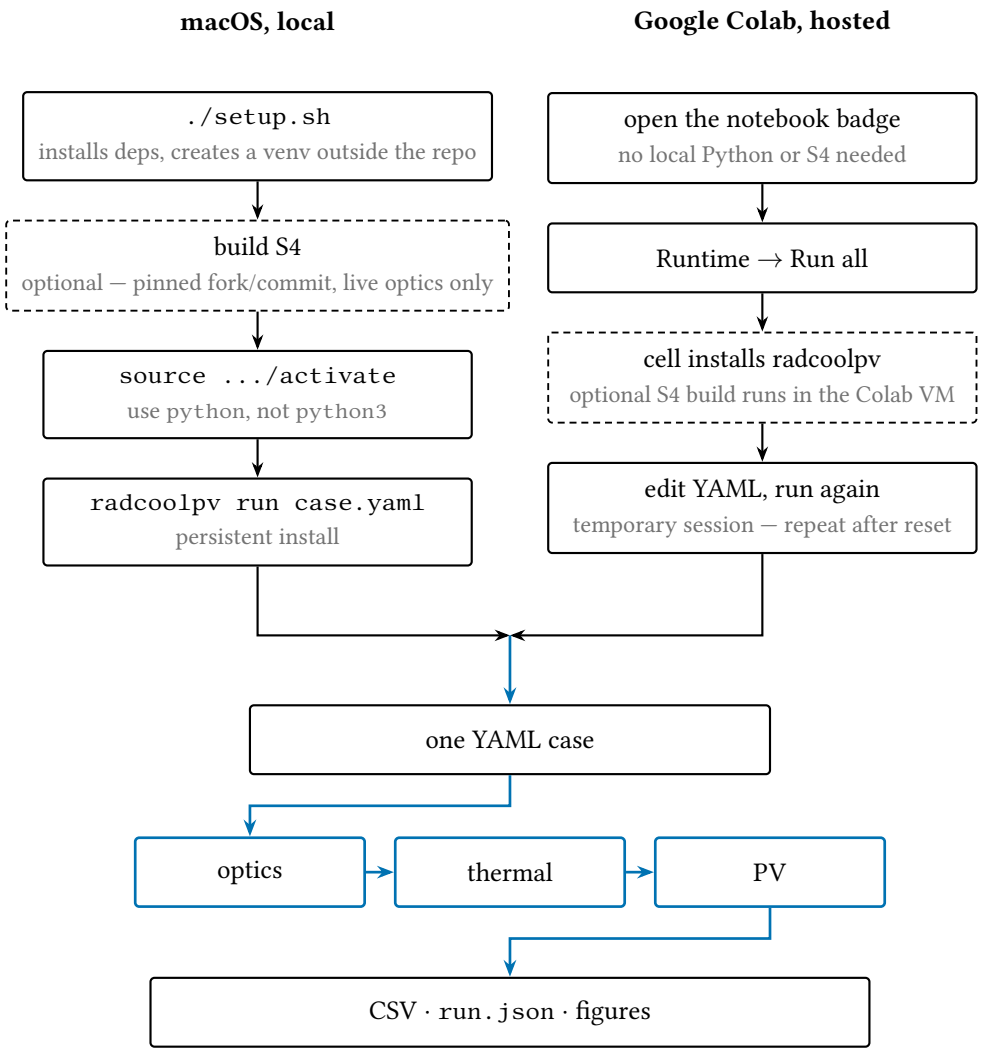

FIGURE 1. The two supported entry points. A local macOS install (Section 2) is a one-time, persistent setup; a Colab notebook needs no local install but resets with the runtime. Both drive the identical YAML case through the same three-stage pipeline (Section 4) to the same outputs (Section 9).

The Colab notebooks require no local Python or S4: opening the notebook badge and choosing **Runtime → Run all** installs `radcoolpv` in the temporary checkout

and, where a case needs it, rebuilds S4 inside the Colab virtual machine. A Colab runtime is temporary — the setup cell must run again after any reset — whereas a local install persists across sessions. Both paths call the identical entry point, `radcoolpv run case.yaml`, on the identical YAML schema described in Section 8, and produce the identical outputs described in Section 9.

Where each quantity is computed

Every equation in this manual is implemented in one place. The table below is the index from physics to source, so a reader checking a result never has to guess which function produced it.

Quantity	Module	Function
YAML parsing, defaults, validation	`config.py`	`from_dict`, `validate`
Stage sequencing	`pipeline.py`	`run`
Layer stack from geometry	`optics/geometry.py`	`build_structure`, `_photonic_layers`
Permittivity per material	`materials/tabulated.py`, `materials/analytic.py`	`_interpolator`, `drude_si3n4`
R, T, A, A_{Si} from S4	`optics/s4_backend.py`	`sweep`, `_fluxes`
Angular quadrature nodes	`config.py` (`SimulationConfig`)	`directions`
Directional → hemispherical	`optics/directional.py`	`reduce`, `_selected`
Resuming a stored spectrum	`optics/directional.py`	`from_reduced_file`
Band averages	`optics/averages.py`	`band_average`, `pv_band_averages`
Solar spectrum, photon flux	`thermal/spectra.py`	`load_solar`, `load_atmosphere`
Planck-weighted radiated power	`thermal/radiative.py`	`rad_power`
Band gap, J_{sc}, J_0, Auger, I–V	`thermal/pv.py`	`solve_iv`
Luminescence term	`thermal/pv.py`	`non_thermal_power`
MPP and V_{oc} refinement	`thermal/pv.py`	`refine_peak`, `open_circuit_voltage`
Energy balance, T_{eq}	`thermal/energy_balance.py`	`run`, `_zero_crossing`
CSV, export, manifest	`io/clean_writers.py`	`_write_csv`, `write_optics_export`, `write_run_json`

Optical model

The S4 problem

The YAML structure is ordered from the incident medium toward one semi-infinite terminal layer. S4 expands the in-plane fields in a Fourier basis of size `simulation.s4_modes`. Flat, circular, rectangular, and polygonal regions are supplied to S4 analytically; spheres and hemispheres use the configured stack of circular slices.

At each wavelength, direction, and polarization, the code evaluates the integrated normal Poynting flux. With incident flux P_{inc},

$$R = -\frac{P^{-}_{\mathrm{top}}}{P_{\mathrm{inc}}}, \tag{1}$$

$$T = \frac{P^{+}_{\mathrm{bottom}}}{P_{\mathrm{inc}}}, \tag{2}$$

$$A = 1 - R - T. \tag{3}$$

Unlike the old Lua output, T is normalized by incident flux at oblique incidence. The silicon-layer absorptance uses the net flux at the two interfaces of the silicon layer itself:

$$A_{\mathrm{Si}} = \frac{P_{\mathrm{net}}(z_{\mathrm{Si,top}}) - P_{\mathrm{net}}(z_{\mathrm{Si,bottom}})}{P_{\mathrm{inc}}}. \tag{4}$$

This prevents absorption in a layer below silicon from being assigned to the PV cell.

Because A is formed as $1 - R - T$, energy conservation is structural rather than a diagnostic: what it tests is the flux normalisation, not the solver. The residual $|R + T + A - 1|$ computed in memory is at machine precision ($\sim 10^{-16}$); the $\sim 10^{-7}$ seen in stored spectra is the `%.6e` text format, not a physical error.

Unit cell and shape discretisation

`geometry.lattice.type` selects the Bravais cell. `square` builds vectors $(x, 0), (0, x)$ and *ignores* `lattice.y`. `hexagonal` builds $(x, 0), (0, y)$ and additionally places a second copy of every circular motif at the cell centre $(x/2, y/2)$: the hexagonal array is represented as a centred-rectangular supercell with two motifs. For nearest-neighbour pitch p, the correct entries are

$$x = \sqrt{3}\, p, \qquad y = p, \tag{5}$$

which places all nearest neighbours at distance p. The centred motif is added only for `cylinder`, `sphere`, and `semisphere`; for `triangle` and `grating` a hexagonal setting changes the cell vectors but yields a single motif, so those shapes are effectively rectangular.

A centred-rectangular cell with aspect ratio $\sqrt{3} : 1$ and a mid-cell motif is the standard representation of a triangular (hexagonal) lattice — not a coincidence of this code.

Curved shapes are stacks of uniform slices, since S4 layers are z-invariant. For a sphere or hemisphere of radius a discretised into $N =$ `discretization_layers` slices of thickness $\delta = 2a/N$, slice i is a circle of radius

$$r_i = \sqrt{\max(a^2 - (y_i - a)^2,\ 0)}, \qquad y_i = 2a - \tfrac{\delta}{2} - i\,\delta. \tag{6}$$

A hemisphere keeps the top $\lfloor N/2 \rfloor$ whole slices and, for odd N, one half-thickness slice straddling the equator, so the dome spans exactly a for either parity. A triangle of base b and height H uses rectangles of half-width $b\,y/2H$. `grating` is a one-dimensional lamellar layer: ridges of `photonic_material` with period `lattice.x`, duty r, and a vacuum groove of half-width $(1 - r)\,p/2$ spanning the full cell in y.

N is a convergence parameter, not a free choice: a slice stack is a staircase approximation of a curved surface, and only `cylinder`, `grating`, and `flat` are represented exactly.

Directions and polarization

One YAML case supports one of three modes:

- `normal`: $\theta = 0$;
- `specific`: one user-defined polar angle θ and azimuth ϕ;
- `hemispherical`: a two-dimensional (θ, ϕ) quadrature.

`polarization` is `TE`, `TM`, or `unpolarized`. TE and TM are never assumed equal solely because $\theta = 0$; anisotropic unit cells such as one-dimensional gratings can give different normal-incidence responses. Unpolarized output is

$$X_{\text{unpol}} = \frac{X_{\text{TE}} + X_{\text{TM}}}{2}, \qquad X \in \{R, T, A, A_{\text{Si}}\}. \tag{7}$$

For one polarization p, the hemispherical property is

$$X_{\text{h},p}(\lambda) = \frac{1}{\pi} \int_0^{2\pi} \int_0^{\pi/2} X_p(\lambda, \theta, \phi) \cos\theta \sin\theta \, d\theta \, d\phi. \tag{8}$$

The implementation applies Gauss–Legendre quadrature in $u = \sin^2\theta$ and a uniform periodic azimuth rule. It avoids the grazing endpoint and includes a separate zero-weight normal probe for the PV luminescence term. Quadrature weights sum to one. A true hemispherical result therefore requires convergence with respect to both `hemisphere_theta_points` and `hemisphere_azimuth_points`.

Emittance and Kirchhoff's law

The code labels total absorptance as emittance using directional Kirchhoff reciprocity,

$$\epsilon_p(\lambda, \theta, \phi, T) = A_p(\lambda, \theta, \phi, T). \tag{9}$$

The equality is per direction, wavelength, polarization, and temperature — not a single scalar statement that a surface simply 'is' some one emittance.

This identification requires a passive reciprocal structure in local thermal equilibrium, evaluated with the same wavelength, direction, polarization, and temperature-dependent material state. It is not automatically valid for nonreciprocal, active, nonlinear, or nonequilibrium emitters. The present material models are temperature-independent.

Materials and numerical validity

Tabulated models interpolate n and k linearly and use $\epsilon = (n+ik)^2$. Requests outside a table's wavelength interval raise an error; endpoint clamping and silent extrapolation are forbidden. The Akerboom case uses the unmodified refractiveindex.info Olmon evaporated-Au record. Its SiO2 and Si refractive-index tables are inherited from an earlier implementation: the former is labelled Palik/Kitamura; the latter has no bibliographic source and is made lossless to implement the paper's explicit nonabsorbing-Si assumption. The current refractiveindex.info database has no records labelled with the paper-cited Palik/Kitamura Si or SiO2 sources. This provenance gap is retained as a limitation rather than silently replacing the models.

Every scientific result must demonstrate:

1. passivity and finite R, T, A, A_{Si}, with $R + T + A \approx 1$;
2. convergence in S4 Fourier basis size;
3. for hemispherical results, convergence in both angular counts.

Atmosphere, thermal balance, and PV model

Atmospheric and thermal radiation

For zenith transmittance $\tau_{\mathrm{atm}}(\lambda)$, the directional atmospheric emittance is

$$\epsilon_{\mathrm{atm}}(\lambda, \theta) = 1 - \tau_{\mathrm{atm}}(\lambda)^{1/\cos\theta}. \tag{10}$$

The surface–atmosphere product is integrated with the same angular quadrature as the surface emittance. With Planck spectral radiance $B_\lambda(T)$,

$$P_{\mathrm{rad}}(T) = \pi \int \epsilon_{\mathrm{h}}(\lambda) B_\lambda(T)\, d\lambda, \tag{11}$$

$$P_{\mathrm{atm}}(T_{\mathrm{amb}}) = \pi \int \langle \epsilon \epsilon_{\mathrm{atm}} \rangle_{\mathrm{h}}\, B_\lambda(T_{\mathrm{amb}})\, d\lambda, \tag{12}$$

$$P_{\mathrm{conv}}(T) = h\,(T - T_{\mathrm{amb}})\,. \tag{13}$$

The convection coefficient h is the total effective nonradiative coefficient for the reference area used by the balance. It is an external lumped-model input and is not deduced from geometry or weather data. A per-surface coefficient must be combined explicitly by the user; the library does not multiply h by an assumed number of exposed surfaces.

h is a single lumped number for everything non-radiative: convection and any other loss the user has not separated out.

`rad_power` returns the radiance integral $\int \epsilon(\lambda) B_\lambda(T)\,\mathrm{d}\lambda$ in $\mathrm{W\,m^{-2}\,sr^{-1}}$, with

$$B_\lambda(T) = \frac{2hc^2}{\lambda^5} \left[\exp\!\left(\frac{hc}{\lambda k_B T} \right) - 1 \right]^{-1}, \tag{14}$$

evaluated on the micrometre grid. The caller multiplies by π to integrate the cosine-weighted hemisphere, so $\pi \int B_\lambda\,\mathrm{d}\lambda$ over a broad band reproduces σT^4; that identity is a useful check on any new wavelength range. Because the integral is trapezoidal on the configured grid, a range that truncates the Planck peak silently understates P_{rad}: at 300 K the peak lies near 9.7 μm and a 4–30 μm window is the usual minimum.

Solar absorption and steady state

Total absorbed solar power is

$$P_{\mathrm{sun}} = \int A(\lambda) I_{\mathrm{AM1.5G}}(\lambda)\, d\lambda. \tag{15}$$

For a literature cooling curve that prescribes absorbed solar power directly, the `absorbed_solar_power` key in the `thermal` section overrides this integral. The Akerboom case uses the paper value 808 $\mathrm{W\,m^{-2}}$.

The cooling-power convention is

$$P_{\mathrm{cool}}(T) = P_{\mathrm{rad}} - P_{\mathrm{atm}} + P_{\mathrm{conv}} - P_{\mathrm{sun}} + P_{\mathrm{MPP}} + P_{\mathrm{nt}}, \tag{16}$$

where P_{MPP} is electrical power removed from the cell and P_{nt} is the non-thermal luminescence term. Equilibrium satisfies $P_{\mathrm{cool}}(T_{\mathrm{eq}}) = 0$.

Cooling-curve cases specify their temperature interval in YAML. Standard PV cases evaluate T_{amb} through $T_{\mathrm{amb}} + 150$ K. If the balance does not bracket a root, the run raises an error. It never reports a sweep endpoint as an equilibrium temperature.

I–V calculation

The short-circuit current uses IQE, silicon-layer absorptance, and the solar photon flux up to the silicon bandgap:

$$J_{\mathrm{sc}} = q \int_0^{\lambda_g} \mathrm{IQE}(\lambda) A_{\mathrm{Si}}(\lambda) \Phi_{\mathrm{sun}}(\lambda)\, d\lambda. \tag{17}$$

At each T, V, the current density J solves

$$0 = -J + \frac{V - JR_s}{R_{\mathrm{sh}}} + J_0(T)\left[\exp\left(\frac{q(V - JR_s)}{k_B T}\right) - 1\right] + J_{\mathrm{Auger}}(T, V) - J_{\mathrm{sc}}. \tag{18}$$

Output power is $-JV$. Each voltage is solved with `scipy.optimize.fsolve`, warm-started from the previous voltage's solution, which keeps the strongly exponential diode equation in its convergence basin across the sweep.

The saturation current is itself a spectral integral rather than a fitted constant, using the same IQE and silicon absorptance against a blackbody photon flux at the cell temperature:

$$J_0(T) = q \int_0^{\lambda_g} \mathrm{IQE}(\lambda) A_{\mathrm{Si}}(\lambda) \Phi_{\mathrm{bb}}(\lambda, T)\, \mathrm{d}\lambda. \tag{19}$$

Auger recombination uses the intrinsic carrier concentration

$$n_i = 5.29 \times 10^{19} \left(\frac{T}{300}\right)^{2.54} \exp\left(\frac{-6726}{T}\right) \ \mathrm{cm}^{-3}, \tag{20}$$

with a coefficient $A(T)$ interpolated from five tabulated points (195–372 K, 3.03–4.55×10^{-31} cm^6/s), giving

$$J_{\mathrm{Auger}} = 2qA(T) n_i^3 d_{\mathrm{Si}} \exp\left(\frac{3qV}{2k_B T}\right). \tag{21}$$

Extrapolating outside that temperature range flattens to the endpoint values, so results far outside 195–372 K should be treated as unsupported.

The band-gap wavelength preserves a scalar-reduction convention from an earlier implementation: it evaluated $E_{g0} - \alpha T^2/(T + \beta)$ with matrix right division on a row vector, which collapses to the single scalar $\alpha\,(T^2 \cdot (T + \beta))/((T + \beta) \cdot (T + \beta))$. One λ_g therefore serves the whole temperature sweep. This is a model-compatibility choice, not a temperature-resolved band-gap model, and it sets the upper limit of every spectral integral above.

Solving for the operating point

P_{MPP} and the luminescence term depend on V_{mpp}, which depends on the temperature that the balance is trying to find. The code resolves this by fixed-point iteration from $V_{\mathrm{mpp}} = 0.65$ V: assemble the balance, locate T_{eq}, re-evaluate the power curve there, and refine V_{mpp}, stopping once successive values agree to the configured tolerance. Failure to settle raises a `RuntimeWarning` naming the last value rather than silently returning it.

Three numerical choices matter for reproducibility, and all three depart deliberately from an earlier grid-based implementation:

- **Equilibrium temperature** is the linearly interpolated zero crossing of P_{cool}, not the nearest grid point. A balance that never crosses zero, or is already non-negative at the lowest swept temperature, raises an error naming the swept interval. An endpoint is never reported as a solution.
- **MPP and** V_{mpp} come from a three-point parabolic fit through the largest sample and its neighbours, so they are not quantised to the `thermal.voltage` step. It falls back to the raw sample at a boundary or for collinear points.
- V_{oc} is the interpolated crossing of $-J$. A prior grid-search approach returned the last grid point before the crossing, biasing V_{oc} low by up to one step (7 mV on the default sweep).

Every equilibrium quantity is then reported at that same interpolated temperature, so T_{eq}, P_{MPP}, V_{oc}, and the fill factor describe one consistent operating point.

All three choices trade a small, deliberate departure from an earlier grid-based implementation for an answer that does not depend on where the sweep happened to land a grid point.

Temperature reduction

When `thermal.reference_temperature` is supplied,

$$\Delta T_{\mathrm{reduction}} = T_{\mathrm{reference}} - T_{\mathrm{eq}}. \tag{22}$$

The user must define the reference physically, for example the operating temperature of a bare module under the same boundary conditions. The library does not invent or silently substitute a reference.

Numerical methods and their failure modes

The diode row hides a distinction worth stating. MINPACK reports “not making good progress” whenever a step stops improving the residual, which is also what happens once the residual has reached the floating-point floor and there is nothing left to improve. Called without `full_output`, `fsolve` turns both into the same warning

Step	Method	Fails when
Polar quadrature	Gauss–Legendre in $u = \sin^2\theta$	too few nodes for a sharply directional emitter
Azimuth quadrature	uniform periodic rule	anisotropic cells with few azimuth points
Material n, k	linear interpolation, extrapolation refused	requested λ outside the table (raises)
Spectral integrals	trapezoidal on the configured grid	grid truncates the Planck peak or undersamples the solar band
Band averages	exact band, endpoints interpolated	band not spanned by the grid (omitted, never zero)
Diode I–V	`fsolve`, warm-started per voltage, accepted on its residual	a residual that stays large (raises)
MPP, V_{mpp}	three-point parabolic refinement	peak on a sweep boundary (falls back to the sample)
V_{oc}	interpolated $-J$ zero crossing	sweep does not bracket it (raises)
T_{eq}	interpolated P_{cool} zero crossing	no crossing in range (raises, never returns an endpoint)
V_{mpp} coupling	fixed-point iteration	does not settle (warns, naming the last value)

and returns its answer either way, so a solve exact to 10^{-14} is indistinguishable from one that failed — and the failed one's wrong current flows into the results. `_solve_diode` takes the status itself and judges by the residual: negligible against J_{sc} is accepted, anything else raises. Across a converged group C case, one solve in roughly ninety thousand reports the stall, with a residual of -5.7×10^{-14}.

The band-average row is worth dwelling on, because the obvious implementation fails quietly rather than loudly. A legacy convention located each band edge by taking the first sample within a fixed ± 0.015 μm of it; on a grid with no sample that close to 1.12, 4, 8, 13, 17 or 24 μm the average returned 0.0 rather than raising, and that zero reached `run.json` as though it were a result. Whether it happened was essentially luck: on a 0.3–24.9 μm grid, $n = 400$ finds four of the six edges, $n = 800$ finds five, and only $n = 1600$ finds all six.

Choosing a wavelength range freely is only safe once the band averages stop depending on where the samples happen to land.

Both `band_average` and `pv_band_averages` now integrate exactly the band requested, interpolating the two endpoints onto the grid, so the value does not depend on whether a sample lands on an edge. A band the grid does not span returns `None` and is omitted from the manifest. A missing key therefore means "not covered by this grid", never "zero". This is what lets the wavelength range be chosen for the problem at hand rather than for the band edges.

YAML examples

Configuration key reference

The `run` block selects which stages execute:

- `optics: true` runs the optical simulation (S4 or the freeform solver) to compute the spectral reflectance/emittance; `false` skips it, and `run.optics_results` must then point at a stored spectrum, since the thermal stage needs one to consume: a five-column `lambda, R, T, emit, abs_si` export, a seven-column form, or — with `run.optics_results_emittance_column` — one emittance column of a plain table, which is treated as an opaque surface. E.g. `run: {optics: false}`.
- `thermal: true` solves the atmospheric/thermal energy balance and, for a PV case, the I–V curve, from that spectrum; `false` skips it, leaving an optics-only run (for example, to write a spectrum with `run.optics_export` without solving anything downstream). E.g. `run: {thermal: false}`.
- `plots: true` writes the run's figures alongside its CSV/JSON outputs (Section 9); `false` skips figure generation. E.g. `run: {plots: false}`.

`run.mode` selects the physics variant, and which figures it produces:

- `standard` (default): the full PV path. Absorbed solar power is the band-integrated spectrum against the AM1.5 irradiance, and the energy balance is solved jointly with the I–V curve for equilibrium temperature, MPP, and V_{oc}. *Requires* a `silicon` layer in `structure`: the I–V solve reads its thickness for the Auger term and raises if none is present. E.g. `run: {mode: standard}`.
- `test`: a PV-free cooling curve checked against a literature case (Perrakis Fig. 2). Absorbed solar power is pinned to a fixed 620 W/m^2 test constant instead of the spectrum, and the figure overlays the literature curve. *Requires* `run.thermal: true`; it takes no PV-specific config and produces no output when `run.thermal` is false. E.g. `run: {mode: test, optics: false}`.
- `cooling_curve`: also a PV-free cooling curve, but swept over `thermal.cooling_temperature` (`min`/`max`/`n`) rather than a small window near ambient temperature. Absorbed solar power comes from `thermal.absorbed_solar_power` or the real spectrum, optionally rescaled by `thermal.solar_irradiance`. *Requires* `run.thermal: true` and `thermal.cooling_temperature` with `max > min` and $n \geq 2$; `thermal.solar_irradiance` and `thermal.absorbed_solar_power`, when set, must each be positive.

E.g. `run: {mode: cooling_curve}` with `thermal: {cooling_temperature: {min: 260.0, max: 380.0, n: 121}}`.

- `spectral_compare`: runs neither stage — it requires `run.optics: false` and `run.thermal: false` — and instead overlays the digitized spectra listed in `comparison.spectra` on one comparison figure. *Requires* a non-empty `comparison.spectra`, each entry giving `label` and `file`, and `comparison.xlim` / `comparison.ylim` each holding exactly two values. E.g. `comparison: {spectra: [{label: "Digitized", file: data/ref.txt}]}`.

The remaining `run` keys:

- `results_dir` (default `results`): parent folder for the timestamped output directory (Section 9), resolved relative to the YAML file. E.g. `results_dir: results/my_case`.
- `write_outputs` (default `true`): write the timestamped CSV/JSON folder. `false` is for tests or programmatic use; it does not affect `run.optics_export`, which is written whenever set, by explicit path. E.g. `write_outputs: false`.
- `optics_results`: path to a prior optics folder or reduced spectrum file, read when `optics: false`. *Requires* a value whenever `thermal: true` and `optics: false`. E.g. `optics_results: validation/data/fig5a_measured_emittance.txt`.
- `optics_results_angles` (default `hemispherical`): a provenance label recorded in `run.json`, not a behavioural switch — `normal` and `hemispherical` read the same file identically. *Requires* `normal` or `hemispherical`. E.g. `optics_results_angles: normal`.
- `optics_results_emittance_column`: zero-based column to read as hemispherical emittance from a digitized multi-column table (column 0 is wavelength). *Requires* ≥ 1 when set. E.g. `optics_results_emittance_column: 2`.
- `optics_export`: path to write the resumable five-column spectrum (`lambda_um R T emit abs_si`) for a later `optics_results` to read back. E.g. `optics_export: data/optics/case.txt`.

`simulation.wavelength` sets the spectral grid as `min`/`max`/ `n` in μm, linearly spaced. *Requires* `min` > 0, `max` $>$ `min`, and `n` ≥ 2. E.g. the full-spectrum default, `wavelength: {min: 0.3, max: 30.0, n: 2000}`.

`simulation.angles` selects one of three direction modes (Section 5.3 gives the physics):

- `normal`: one direction at $\theta = 0$, $\phi =$ `azimuth_angle_deg`. E.g. `angles: normal`.
- `specific`: one direction at $\theta =$ `polar_angle_deg`, $\phi =$ `azimuth_angle_deg`. *Requires* `polar_angle_deg` $\in [0, 90)$. E.g. `angles: specific, polar_angle_deg: 35.0, azimuth_angle_deg: 90.0`.
- `hemispherical` (default): Gauss–Legendre quadrature over `hemisphere_theta_points` × `hemisphere_azimuth_points` directions (default $8 \times 12 = 96$), plus one zero-weight normal probe. *Requires* both point counts ≥ 1. A live S4 thermal run additionally requires `angles: hemispherical`: a directional spectrum cannot define the radiative energy balance. E.g. `angles: hemispherical, hemisphere_theta_points: 8, hemisphere_azimuth_points: 12`.

The remaining direction and polarization keys:

- `polar_angle_deg` (default 0): polar angle, read only by `specific`. E.g. `polar_angle_deg: 35.0`.
- `azimuth_angle_deg` (default 0): azimuth, read by `normal` and `specific`. *Requires* $[0, 360)$ in any mode. E.g. `azimuth_angle_deg: 90.0`.
- `polarization`: `TE`, `TM`, or `unpolarized` (default; averages TE and TM equally). E.g. `polarization: TM`.
- `s4_modes` (default 10): S4 Fourier truncation (`NumBasis`). *Requires* ≥ 1; read only when `geometry.source: s4`. E.g. `s4_modes: 80`.

`geometry.*` configures a live S4 sweep; every key below is validated only when `run.optics: true` and `geometry.source: s4` (Section 5.2 gives the shape mathematics). With `run.optics: false` the frozen spectrum at `run.optics_results` governs the result and `geometry`/`structure` are not consulted at all, silicon thickness for the Auger term excepted.

- `source` (default `s4`): `s4` runs a live sweep; `freeform` instead loads a precomputed dataset (λ, total reflectance, total absorptance, …, silicon absorptance in column 6) from `geometry.freeform.file`, interpolated onto `simulation.wavelength.n` points spanning the file's own range. *Requires* `s4` or `freeform`; `freeform` additionally requires `geometry.freeform.file` to be set. E.g. `geometry: {source: freeform, freeform: {file: data/optics_dataset.csv}}`.
- `shape`: `flat`, `sphere`, `semisphere`, `triangle`, `cylinder`, or `grating`. Each non-flat shape reads its parameters from a same-named block, *except* `semisphere`, which reuses `sphere: {radius: ...}`. *Requires*:

sphere/semisphere need radius; triangle needs base and height; cylinder needs radius and height; grating needs duty $\in (0, 1)$ and depth. All lengths must be > 0; grating's period is lattice.x, not a separate key. E.g. shape: cylinder, cylinder: {radius: 1.75, height: 2.25}.

- photonic_material (default sio2): the shape's material key, resolved through materials like every layer. E.g. photonic_material: sio2.
- lattice.type: square (ignores lattice.y) or hexagonal (Section 5.2). lattice.x/lattice.y (default 20 μm): cell vectors. *Requires* both > 0. E.g. lattice: {type: hexagonal, x: 10.608811, y: 6.125}.
- discretization_layers (default 1): slice count approximating a curved shape. *Requires* ≥ 1 for sphere, semisphere, and triangle; unused for cylinder, grating, and flat, which are represented exactly. E.g. discretization_layers: 20.

structure is the ordered layer stack, incident medium first (Section 5.1). Each entry is {material, thickness, terminal} (terminal default false), thickness in μm. *Requires*, for a live S4 sweep: a non-empty list; exactly one layer terminal: true, which must be last and have thickness: 0 (S4 treats it as semi-infinite); every thickness ≥ 0; and at most one layer with material: silicon. Outside a live sweep, only that silicon layer's thickness is read, for the Auger term (Section 6.3). E.g. structure: [{material: silicon, thickness: 250.0}, {material: substrate, thickness: 0.0, terminal: true}].

materials maps every material name used in geometry.photonic_material and structure to a tabulated or analytic model key (Section 5.5). *Requires* every such name to appear here, except vacuum, which is built in. E.g. materials: {sio2: PalikKitamura_SiO2, silicon: Akerboom_Si_lossless}.

thermal.* parameterizes the energy balance and, for standard mode, the I–V solve:

- ambient_temperature (default 298 K) and convection_coefficient (default 12 W/m^{2}K): T_{amb} and the lumped nonradiative coefficient h (Section 6.1). E.g. thermal: {ambient_temperature: 298.0, convection_coefficient: 12.0}.
- voltage: {min, max, n} (default 0.1–0.8 V, 100 points): the I–V sweep (Section 6.3). *Requires* n ≥ 2. E.g. voltage: {min: 0.1, max: 0.8, n: 100}.
- equilibrium (default auto): auto finds T_{eq} and V_{mpp} by fixed-point iteration (Section 6.4); manual instead takes emit_temp and vmpp directly, for MATLAB parity. *Requires* auto or manual. E.g. equilibrium: manual.

- `cooling_temperature: {min, max, n}`: the swept temperature range, read only in `run.mode: cooling_curve` (required there; see above). E.g. `cooling_temperature: {min: 260.0, max: 380.0, n: 121}`.
- `solar_irradiance`: in `cooling_curve` mode, rescales the computed solar power by `solar_irradiance` / AM1.5-total; unused otherwise. *Requires* > 0 when set. E.g. `solar_irradiance: 900.0`.
- `absorbed_solar_power`: in `cooling_curve` mode, overrides the band-integrated solar power with a direct value (Section 6.2), e.g. a paper's stated P_{sun}; unused in `standard` or `test`. *Requires* > 0 when set. E.g. `absorbed_solar_power: 808.0`.
- `reference_curve_file`/`reference_curve_column` (column default 1): a digitized curve overlaid on the `cooling_curve` figure; column 0 is temperature. E.g. `reference_curve_file: data/digitized_curve.txt, reference_curve_column: 2`.
- `reference_temperature`: enables the temperature-reduction report, $\Delta T = T_{\text{reference}} - T_{\text{eq}}$ (Section 6.5). *Requires* $>$ 0 K when set. E.g. `reference_temperature: 360.0`.
- `emit_temp` (default 319 K) / `vmpp` (default 0.6586 V): fixed operating point, read only when `equilibrium: manual`. E.g. `emit_temp: 319.0, vmpp: 0.6586`.

`thermal.pv.*` configures the diode model (Section 6.3):

- `series_resistance` (default 0.00011 $\Omega\,\text{m}^2$) and `shunt_resistance` (default 0.1 $\Omega\,\text{m}^2$): R_s and R_{sh} in the diode equation. E.g. `pv: {series_resistance: 0.00011, shunt_resistance: 0.1}`.
- `bandgap.{eg0, alpha, beta}` (default 1.166, 4.73×10^{-4}, 636): the $E_{g0} - \alpha T^2/(T + \beta)$ band-gap model. E.g. `bandgap: {eg0: 1.166, alpha: 4.73e-4, beta: 636}`.
- `iqe_file` (default bundled `data/siliconIQE.txt`): internal quantum efficiency table for J_{sc} and J_0. E.g. `iqe_file: data/siliconIQE.txt`.

`data.solar_spectrum` and `data.atmosphere` point to the AM1.5 spectrum and the atmospheric transmittance table, each resolved relative to the YAML file, then falling back to the file bundled with the package under the same basename — so a config may name the default file without a path. E.g. `data: {solar_spectrum: data/astmg173.xlsx, atmosphere: data/cptrans_nq_100_15.dat}`.

`comparison.*` configures `run.mode: spectral_compare` (`spectra`, `xlim`, and `ylim`, with their requirements, are given above); `xlabel`, `ylabel`, `title`, and

`output_file` are cosmetic, defaulted for a mid-/long-wave emissivity comparison. E.g. `comparison: {title: "PDMS emissivity", output_file: pdms_comparison.png}`.

One file with multiple cases

A top-level `cases` list lets one YAML file define several named runs. The CLI executes them in order:

```
cases:
  - name: normal_TE
    run: {optics: true, thermal: false}
    simulation:
      wavelength: {min: 8.0, max: 13.0, n: 201}
      angles: normal
      polarization: TE
    # geometry, structure, materials, and data follow
  - name: directional_TM
    run: {optics: true, thermal: false}
    simulation:
      wavelength: {min: 8.0, max: 13.0, n: 201}
      angles: specific
      polar_angle_deg: 35.0
      azimuth_angle_deg: 90.0
      polarization: TM
```

YAML anchors may hold shared sections. The Akerboom case uses one such file for twelve cases and references digitized text data directly; no helper script is needed.

Normal and directional emittance

```
run:
  optics: true
  thermal: false
  plots: false
  write_outputs: true

simulation:
  wavelength: {min: 8.0, max: 13.0, n: 300}
  angles: specific
  polar_angle_deg: 35.0
  azimuth_angle_deg: 90.0
  polarization: TM
  s4_modes: 80
```

For normal incidence, set `angles: normal`; the requested TE, TM, or both are

still computed explicitly.

Hemispherical emittance

```
simulation:
  wavelength: {min: 3.0, max: 30.0, n: 1000}
  angles: hemispherical
  polarization: unpolarized
  hemisphere_theta_points: 8
  hemisphere_azimuth_points: 12
  s4_modes: 100
```

A live S4 thermal case requires `angles: hemispherical`: the energy balance integrates emissivity over the hemisphere, so a single normal-incidence spectrum does not define it. Convergence must be checked by increasing the polar-angle count, azimuth count, and S4 Fourier basis size independently.

That requirement is what makes a live thermal run expensive. The solver cost scales as $n_\lambda \times n_\theta \times n_\phi \times n_{\text{pol}}$: the listing above is 1000 wavelengths over $8 \times 12 = 96$ quadrature directions, plus one zero-weight normal probe, and an unpolarized case evaluates both TE and TM at every one of those points. Changing a convection coefficient or an ambient temperature does not change any of it.

The practical workflow is therefore to run the optics once, write the reduced spectrum, and re-drive the thermal stage from that file with `run.optics_results`, which costs no S4 time. Group B of the Akerboom case is built this way, and so is any run driven by measured data: with `run.optics: false` the S4 module is never imported at all, so the whole thermal and PV chain works on a machine that has no solver built.

To chain the two runs without any intermediate step, set `run.optics_export` in the optics case to a fixed path and point `run.optics_results` at that same path in the thermal case. These are the write and read halves of one six-column format (`lambda_um R T emit abs_si <emit*emit_atm>`).

That sixth column is what makes a resumed run reproduce the run it came from. A live hemispherical sweep forms the atmospheric term as the angular average of $\epsilon_{\text{atm}}(\lambda,\theta)\,\epsilon(\lambda,\theta)$, and no spectrum carries enough information to rebuild it: a reader given only the averaged emittance has to fall back on the zenith atmosphere, which is a different number. Five-column files are still read, with that approximation. The export is written even when `run.write_outputs` is false, because it is requested by explicit path. Note that `optics.csv` cannot be reused here: it is comma-separated with a text header, which the resume reader rejects, and it lands in a timestamped folder that a later case cannot name in advance. `examples/freeform_pv.yaml` is the minimal end-to-end example: optics, energy balance and cell, with no S4.

The sixth column is not a convenience. Without it a resumed hemispherical run solves a different energy balance from the one it resumed.

A resumed spectrum retains no directional information, so the atmospheric term

is angle-independent whatever the file represents. Be aware that `run.optics_results_angles` does not change this, or any other number: it is recorded in `run.json` and in the export header as a statement of what the stored spectrum is, and `normal` and `hemispherical` produce bit-identical results. Driving the thermal stage from a normal-incidence spectrum remains an angle-independent approximation rather than a hemispherical calculation, and must be reported as such — but it is the provenance of the file, not this key, that makes it so.

Layer stack and materials

```
geometry:
  source: s4
  shape: cylinder
  photonic_material: sio2
  lattice: {type: hexagonal, x: 10.608811, y: 6.125}
  cylinder: {radius: 1.75, height: 2.25}

structure:
  - {material: sio2, thickness: 500.0}
  - {material: silicon, thickness: 500.0}
  - {material: gold, thickness: 0.08}
  - {material: vacuum, thickness: 0.0, terminal: true}

materials:
  sio2: PalikKitamura_SiO2
  silicon: Akerboom_Si_lossless
  gold: RII_Olmon_2012_ev_Au
```

Exactly one terminal layer is required, it must be last, and its thickness must be zero because S4 treats it as semi-infinite.

Thermal case and temperature reduction

```
thermal:
  ambient_temperature: 300.0
  # Calibrated effective total coefficient; the paper states 6.0.
  convection_coefficient: 12.54
  absorbed_solar_power: 808.0
  reference_temperature: 360.0
  cooling_temperature: {min: 260.0, max: 380.0, n: 121}
  equilibrium: auto
```

For a digitized table whose first column is wavelength and later columns are different measured emittances, set `run.optics_results_emittance_column`

to the desired zero-based column. Column zero is reserved for wavelength.

Outputs and reproducibility

New runs write:

- `optics.csv`: selected directional or hemispherical spectrum;
- `optics_directional.csv`: every S4 wavelength, polar angle, azimuth, polarization, angular weight, R, T, A, A_{Si};
- `iv.csv`, `power.csv`, or `cooling_power.csv`;
- `run.json`: full resolved configuration, Git revision, dirty flag, interpreter/-platform, S4 binary hash, input paths and SHA-256 hashes, and scalar results;
- figures when `run.plots: true`.

The `run.json` manifest

Five top-level keys. `resolved_config` is the complete configuration after defaults and validation, so a run can be reproduced without the original YAML. `provenance` carries `python`, `platform`, `git_commit`, `git_dirty`, `inputs` (every resolved input path with its SHA-256), and `s4` (module path and hash, `null` when no live optics ran). `optics` records `angles`, `polarization`, `n_lambda`, `wavelength_range_um`, and `silicon_from_emittance` — the last is true when the silicon absorptance was inferred from a supplied emittance column rather than solved for, so a PV result resumed from measured data is never mistaken for a solved one.

`resolved_config` alone is enough to reproduce a run: it is the configuration after every default and validation step has already been applied.

The six power terms are the energy balance at equilibrium, reported so the manifest closes on itself: $P_{\mathrm{rad}} - P_{\mathrm{atm}} + P_{\mathrm{conv}} - P_{\mathrm{sun}} + P_{\mathrm{mpp}} + P_{\mathrm{nt}} = P_{\mathrm{cool}} = 0$.

Only a run that actually solves a cell reports the electrical rows. A PV-free cooling-curve run omits every one of them rather than writing 0.0, which would read as "zero efficiency" instead of "no cell was solved"; the same rule governs `band_averages_percent` below.

Ambient and equilibrium appear as pairs so the cost of running hot reads directly off the manifest. The ambient V_{oc} is the highest in the sweep, so a voltage range that legitimately brackets the operating point may not reach it; that case reports `null` rather than 0.0, and does not stop the run, because the equilibrium point is the result.

J_0 and J_{Auger} exist across the whole (T, V) sweep inside `IVResult` — the Auger array spans the whole sweep — so each is interpolated to T_{eq}, the Auger term additionally at V_{mpp}. The sweeps themselves belong in `iv.csv` and `power.csv`, not in a scalar manifest.

`thermal_results` key	Meaning
`equilibrium_temperature_K`	interpolated root of P_{cool}
`temperature_reduction_K`	$T_{\text{ref}} - T_{\text{eq}}$; `null` without a reference
`vmpp_V`	converged fixed-point MPP voltage
`short_circuit_current_A_per_m2`	J_{sc}
`mpp_ambient_W_per_m2`	MPP at T_{amb}
`mpp_equilibrium_W_per_m2`	MPP at T_{eq}
`voc_ambient_V`	V_{oc} at T_{amb}; `null` if outside the sweep
`voc_equilibrium_V`	V_{oc} at T_{eq}
`fill_factor_ambient`	`null` with `voc_ambient_V`
`fill_factor_equilibrium`	fill factor at T_{eq}
`radiative_power_W_per_m2`	$P_{\text{rad}}(T_{\text{eq}})$
`atmospheric_power_W_per_m2`	P_{atm}
`convective_power_W_per_m2`	$P_{\text{conv}}(T_{\text{eq}})$
`absorbed_solar_power_W_per_m2`	P_{sun}
`non_thermal_power_W_per_m2`	$P_{\text{nt}}(T_{\text{eq}})$
`net_cooling_power_W_per_m2`	P_{cool}, zero at T_{eq}
`temperature_coefficient_perc_per_K`	β_P, negative for silicon
`efficiency_equilibrium`	P_{mpp}/AM1.5 total
`saturation_current_equilibrium_A_per_m2`	$J_0(T_{\text{eq}})$
`auger_current_equilibrium_at_vmpp_A_per_m2`	$J_{\text{Auger}}(T_{\text{eq}}, V_{\text{mpp}})$

A PV run adds a fifth key, `band_averages_percent`, holding `solar_absorptance_silicon`, `solar_reflectance`, `subgap_reflectance`, `emittance_8_13um` and `emittance_broadband`: the weighted averages of Section 7, taken against AM1.5 below the gap and against the blackbody at T_{eq} above it. A PV-free `cooling_curve` run has no operating point to weight them at and omits the key rather than reporting zeros. `emittance_broadband` runs from 4 μm to the end of the grid, so read it together with `optics.wavelength_range_um`. A band the grid does not span is omitted, never reported as 0.0: a missing key means "not covered by this grid", never "zero".

Set `run.write_outputs: false` for tests or programmatic validation. Historical reference files from the earlier implementation remain only as read-only parity fixtures; they are never emitted by the active package.

Validation evidence

There is one literature case, reproduced in three groups: the optics, the cooling balance, and the full cell. The optics agree closely. The cooling balance is sensitive to a coefficient the paper reports but that this model cannot check independently, so that group is presented with both values rather than only the one that matches.

Akerboom, Doeleman, Scherer, Zeman, Smith, Isabella and Garnett, "Passive Radiative Cooling of Silicon Solar Modules with Photonic Silica Microcylinders", *ACS Photonics* **9** (2022) 3831–3840, doi:10.1021/acsphotonics.2c01389.

Evidence label is doing real work: reproduction, digitized comparison, and calibrated fit are not interchangeable claims. This case supplies one of each.

A hexagonal array of silica microcylinders — radius 1.75 μm, height 2.25 μm, pitch 6.125 μm — on 500 μm silica over 500 μm silicon over 80 nm gold, with the bare

module and an unpatterned silica slab as references. The gold blocks transmission, so the emissivity is $1 - R$. `validation/akerboom.yaml` defines twelve cases in three groups:

```
radcoolpv run validation/akerboom.yaml
radcoolpv run validation/akerboom.yaml --case
    B3_cooling_h6_cylinders
```

Group B needs no solver. Groups A and C call S4 and cost roughly four minutes and fifty minutes at the converged settings in the file, so their spectra were computed once and committed beside the digitized traces as `validation/data/<case>.txt`. Each is written by the case that names it, through `run.optics_export`, so running that case regenerates it. The notebooks read them, which is how each reproduces its table in seconds on a machine with no solver. A case that cannot run does not abort the others.

Group A: optics

Normal-incidence emittance, 2–16 μm in steps of 0.05 μm, 60 Fourier modes, against Figure 3a. Silicon is modeled as nonabsorbing, which is the paper's own stated assumption for the cooling band. Mean emittance over 7.5–16 μm:

Surface	radcoolpv	Digitized	Paper text
Bare Au/Si	0.032	0.036	~3.5%
Flat silica	0.842	0.843	—
Silica cylinders	0.984	0.977	—

The optics agree.

Group B: cooling curve

This group runs the thermal model alone, driven by the *measured* emittance digitized from Figure 5a. No geometry, no materials, no solver. The isolation is deliberate: what this group tests is the energy balance, with the optics held fixed at measured values. With the coefficient the Methods state, $h = 6.0\ \mathrm{W\,m^{-2}\,K^{-1}}$:

Surface	radcoolpv	Paper Fig. 5b
Bare Au/Si	415.4 K	360 K
Flat silica	360.6 K	339 K
Silica cylinders	355.6 K	336 K

One least-squares fit to all three digitized curves gives $h = 12.54\ \mathrm{W\,m^{-2}\,K^{-1}}$, which reproduces 359.7/340.1/337.5 K. Fitting one coefficient to the curves being

reproduced is a calibration and cannot validate the model that produced them, so cases B4–B6 exist to keep the fit labeled rather than folded silently into the default.

The zero-emitter line needs no optics at all, which is what makes it useful: it bounds every possible emitter from above.

The reason the equilibrium moves so much is that h is a single lumped coefficient standing for all non-radiative exchange, and its value depends on mounting, wind speed and how many surfaces are counted as exchanging heat. A limiting case bounds the whole family without any optics: a surface that does not radiate settles at

$$T = T_{\mathrm{amb}} + \frac{P_{\mathrm{sun}}}{h},$$

which is 434.67 K at $h = 6$ and 364.4 K at $h = 12.54\ \mathrm{W\,m^{-2}\,K^{-1}}$. Every real emitter lands below the line for its own h, so this is worth evaluating before reading any equilibrium temperature. `tests/test_validation_akerboom.py` pins both sets of numbers.

Group C: full optical, thermal and electrical result

0.3–24.9 μm hemispherical with the lossy silicon table. The upper limit is set by gold: `RII_Olmon_2012_ev_Au` is tabulated to 24.93 μm and the loader refuses to extrapolate.

This group requires the lossy `Palik_Si`, not the `Akerboom_Si_lossless` used by groups A and B. Lossless silicon absorbs no sunlight at all, so the photocurrent integrates to zero and the cell reports a few millivolts — a failure that looks like a solver bug and is a materials choice.

Surface	T_{eq}	Efficiency	P_{mpp}	β_P
Bare Au/Si	350.5 K	14.17%	142.6 W m^{-2}	−0.303 %/K
Flat silica	329.3 K	18.09%	182.1 W m^{-2}	−0.299 %/K
Silica cylinders	327.0 K	18.64%	187.7 W m^{-2}	−0.300 %/K

The temperature drops agree with the paper: 21.2 K bare to flat silica against 21 K, 2.3 K flat to cylinders against 3 K, and 23.5 K bare to cylinders against 24 K.

The efficiency column is the one to read carefully: two distinct mechanisms move it, and they are separable.

Two mechanisms drive the efficiency, and the numbers separate them. Absorbed sunlight rises from 507.5 to 598.6 to 614.3 W m^{-2} across the three surfaces, so the silica is acting as an antireflection coating as well as a radiative cooler; the temperature drop and the extra photocurrent contribute separately to the gain.

Standing limitations

The optics are validated against the published spectra. The thermal model is not independently validated by this case. This case has not received a fresh Fourier-mode convergence sweep at the settings quoted above, and the silica cases carry a 2–16 μm residual because the finite stack differs from the paper's semi-infinite-silica

absorption convention.

Required verification for new cases

For every new publication or device case:

1. place all geometry, materials, directions, polarization, wavelength, and numerical settings in YAML;
2. record source data and reject unavailable parameters rather than inventing them;
3. verify $R + T + A$, passivity, material wavelength coverage, and layer-resolved absorption;
4. demonstrate wavelength, Fourier-mode, polar-angle, and azimuth convergence as applicable;
5. distinguish unit tests, legacy-implementation parity, independent optical validation, literature reproduction, digitized comparison, and calibrated fit;
6. compare published and calculated values with absolute or relative errors and retain unresolved failures.

References

[1] V. Liu and S. Fan, "S4: A free electromagnetic solver for layered periodic structures," https://web.stanford.edu/group/fan/S4/.

[2] B. Zhao et al., "Radiative cooling of solar cells with micro-grating photonic cooler," *Renewable Energy* 191, 662–668 (2022), https://doi.org/10.1016/j.renene.2022.04.063.

[3] E. Akerboom et al., "Passive Radiative Cooling of Silicon Solar Modules with Photonic Silica Microcylinders," *ACS Photonics* 9, 3831–3840 (2022), https://doi.org/10.1021/acsphotonics.2c01389.

[4] R. L. Olmon et al., "Optical dielectric function of gold," *Physical Review B* 86, 235147 (2012), https://doi.org/10.1103/PhysRevB.86.235147.

[5] M. N. Polyanskiy, "Refractiveindex.info database of optical constants," *Scientific Data* 11, 94 (2024), https://doi.org/10.1038/s41597-023-02898-2.